TOP 30 BACKEND CONCEPTS

Every Developer Should Know These Concepts

1. What is Backend ?

Backend is the server-side part of an application that handles business logic, database operations, security, and communication.

What it does :

- Processes client requests
- Applies business logic
- Stores & retrieves data from database
- Returns response to the client
- Handles security, authentication, and more

Examples

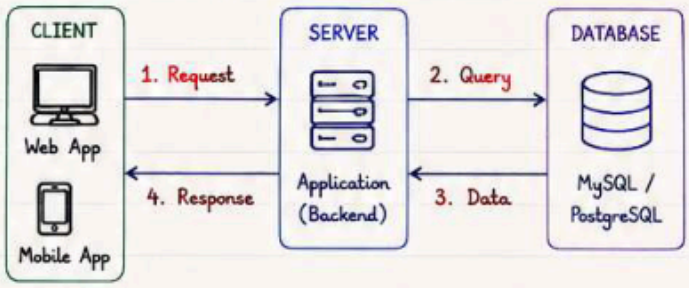
- When you login → backend validates your credentials
- When you place an order → backend saves it in database
- When you request data → backend processes & sends it back

2. Client-Server Architecture

It is a model where client (e.g. browser, mobile app) sends request to server and server sends response.

How it works :

① Client sends a request
② Server processes the request
③ Server interacts with database (if needed)
④ Server sends the response back
⑤ Client displays the result



Example : You open an e-commerce app → browse products → backend gets data from DB → returns list → shows to you.

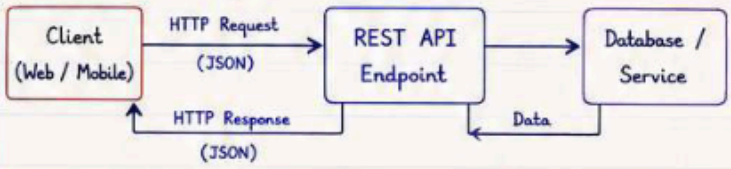
3. REST API

REST (Representational State Transfer) is an architectural style for building APIs using HTTP methods.

Key Characteristics :

- ✓ Stateless
- ✓ Client-Server
- ✓ Cacheable
- ✓ Uniform Interface
- ✓ Layered System
- ✓ Code on Demand (optional)

Typical REST API Flow



Example Request :

```
GET /api/v1/users/123 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token>
```

Example Response :

```
{
  "id": 123,
  "name": "Aman",
  "email": "aman@example.com"
}
```

4. HTTP Methods & Status Codes

HTTP Methods (What we can do)

Method	Use	Example
GET	Retrieve data	/api/users
POST	Create new resource	/api/users
PUT	Update entire resource	/api/users/1
PATCH	Update partial resource	/api/users/1
DELETE	Delete resource	/api/users/1

HTTP Status Codes (How server responds)

2xx – Success
200 OK → Request successful

3xx – Redirection
301 Moved Permanently → Resource moved

4xx – Client Error
404 Not Found → Resource not found

5xx – Server Error
500 Internal Server Error → Something went wrong on server

Quick Example

Request :
GET /api/users/1

Response :
200 OK
{ "id": 1, "name": "Aman" }
or
404 Not Found
{ "message": "User not found" }

☆ Tip : Backend is all about processing requests, managing data, ensuring security and providing reliable & fast responses.

TOP 30 BACKEND CONCEPTS

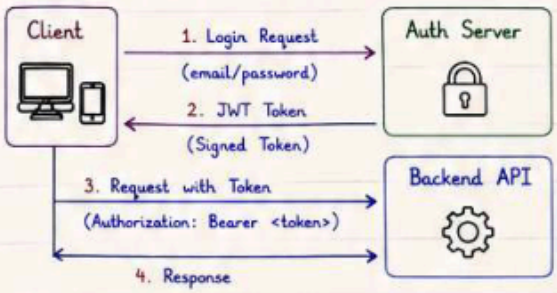
Every Developer Should Know These Concepts

5. API Authentication & Authorization

Authentization verifies who the user is.

Authorization checks what the user can do.

How it works (JWT Example)



Common Methods

- JWT (JSON Web Token)
- OAuth 2.0
- API Key
- Session Based

Example Request

```
GET /api/v1/profile HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGci...
```

Example Response

```
{
  "id": 1,
  "name": "Aman",
  "role": "admin"
}
```

⭐ Pro Tip : Always use HTTPS, store tokens securely, set expiry time for tokens, and follow least-privilege access (give only what's needed).

6. API Versioning

API versioning helps manage changes without breaking existing clients.

Common Approaches

- URI Versioning : /api/v1/users
- Query Parameter : /api/users?version=1
- Header Versioning : Accept: application/vnd.app.v1+json

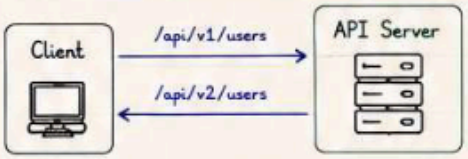
Why Versioning ?

- ✓ Backward compatibility
- ✓ Safe changes
- ✓ Better maintainability

Example

v1 → GET /api/v1/users

v2 → GET /api/v2/users



⭐ Pro Tip : Add version in the URL (v1, v2...) and communicate changes clearly in your API documentation.

7. Pagination

Pagination helps in fetching large datasets in small chunks.

Types

- Offset Pagination (page, limit)
- Cursor Pagination (cursor, limit)

Offset Pagination Example

GET /api/v1/products?page=2&limit=10

Response

```
{
  "data": [ ...10 items... ],
  "page": 2,
  "limit": 10,
  "total": 120,
  "totalPages": 12
}
```

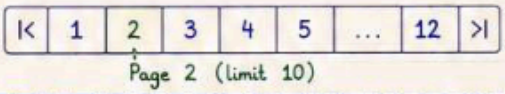
Cursor Pagination Example

GET /api/v1/products?cursor=eyJpZCI6MTIwfQ&limit=10

Response

```
{
  "data": [ ...10 items... ],
  "nextCursor": "eyJpZCI6MjAxfQ",
  "hasMore": true
}
```

⭐ Pro Tip : Use cursor pagination for real-time data or large datasets. It performs better than offset pagination.



8. Filtering, Sorting & Searching

These features help clients get only the data they need.

1) Filtering (where condition)

GET /api/v1/users?role=admin&status=active

2) Sorting (order by)

GET /api/v1/users?sort=name&order=asc

(asc or desc)

3) Searching (keyword search)

GET /api/v1/users?search=aman

Combination Example

GET /api/v1/users?role=admin&status=active&sort=name&order=asc&page=1&limit=10

Example Response

```
{
  "data": [
    { "id": 1, "name": "Aman", "role": "admin" },
    { "id": 2, "name": "Neha", "role": "admin" }
  ],
  "total": 25
}
```

Best Practices

- Use indexing for search fields
- Avoid expensive wildcard searches
- Sanitize and validate all inputs

⭐ Pro Tip : Always validate query parameters. Set max limit (e.g., 100) to prevent server overload.

TOP 30 BACKEND CONCEPTS

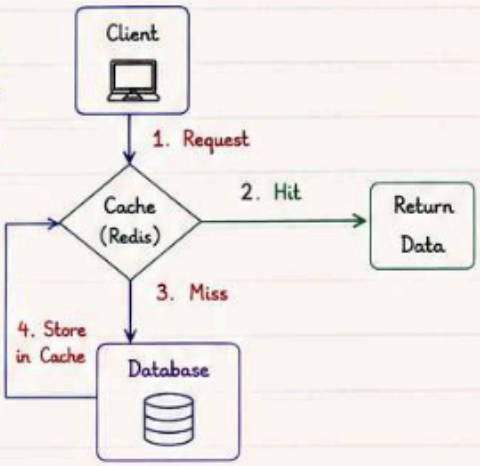
Every Developer Should Know These Concepts

9. Caching

Caching stores frequently used data in memory so that it can be served faster without hitting the database every time.

How it works :

① Client requests data
② Check cache
③ If hit → return data
④ If miss → get data from DB, store in cache, then return



Example (User Profile)

- User requests profile
- If profile is in cache → return
- If not → fetch from DB, store in cache with TTL (Time To Live)

Cache Hit vs Miss

Type	Time
Cache Hit	~1 ms
Cache Miss	~10-100 ms+

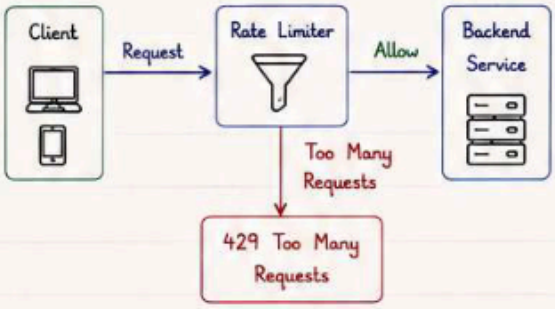
Pro Tip : Choose correct TTL. Too low → more DB calls, too high → stale data.

10. Rate Limiting

Rate limiting restricts the number of requests a client can make in a given time window.

Common Algorithms :

- Fixed Window Counter
- Sliding Window Log
- Token Bucket
- Leaky Bucket



Example

Limit : 100 requests / minute

If user exceeds limit → return 429 status.

HTTP Status

Code	Meaning
429	Too Many Requests

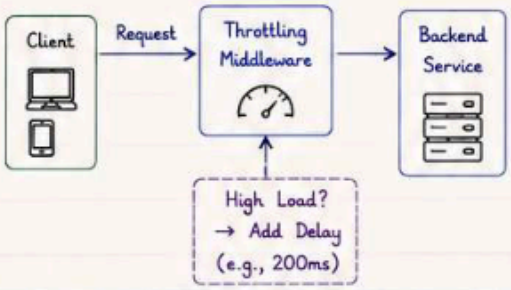
Pro Tip : Apply rate limiting at API gateway / load balancer level for better control and performance.

11. API Throttling

Throttling is similar to rate limiting but focuses on controlling system load by slowing down requests instead of blocking them.

How it works :

- Monitor system health (CPU, memory, etc.)
- If load is high → slow down the requests
- If load is normal → allow full speed



Example

- When traffic is high, add delay in responses.
- Keeps the system stable and prevents crashes.

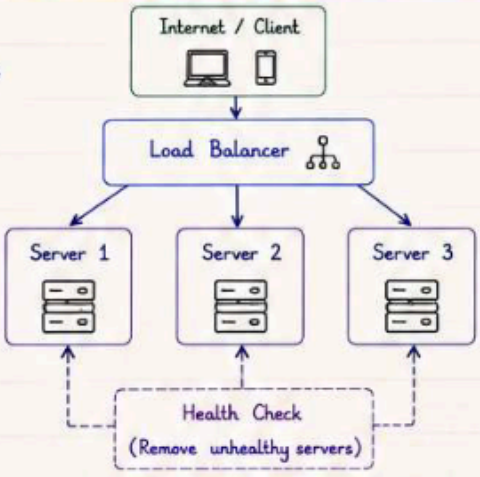
Pro Tip : Throttling protects your backend when traffic spikes unexpectedly.

12. Load Balancer

A load balancer distributes incoming traffic across multiple servers to ensure high availability and scalability.

How it works :

① Client sends request
② Load balancer receives request
③ LB forwards request to one of the healthy servers
④ Server processes and sends response back through LB to client



Algorithms

- Round Robin
- Least Connections
- IP Hash
- Weighted Round Robin

Benefits

✓ High Availability
✓ Scalability
✓ Fault Tolerance
✓ Better Performance

Pro Tip : Always enable health checks so Load Balancer sends traffic only to healthy servers.

TOP 30 BACKEND CONCEPTS

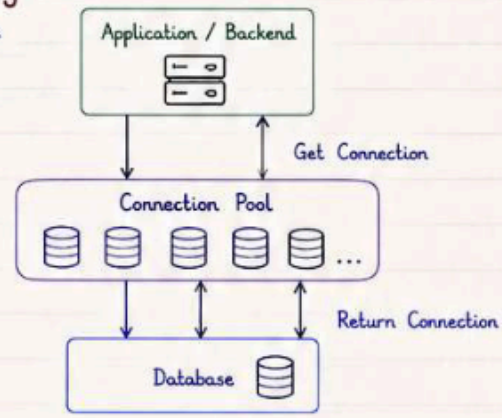
Every Developer Should Know These Concepts

13. Database Connection Pooling

Connection pooling reuses existing database connections instead of creating a new connection for every request.

How it works :

1. Application requests a connection
2. Pool provides an available connection
3. Application uses the connection
4. Connection is returned to the pool



Benefits

- ✓ Reduces connection creation time
- ✓ Improves performance
- ✓ Controls the number of connections to DB
- ✓ Prevents database overload

Example (HikariCP in Java)

- maxPoolSize = 20
- minIdle = 5
- connectionTimeout = 30000ms

☆ Pro Tip : Use connection pooling for high traffic applications to improve response time and reduce database load.

14. SQL vs NoSQL

Different types of databases for different needs.

SQL (Relational)

- Structured data (tables)
- Fixed schema
- ACID compliant
- Uses SQL
- Examples: MySQL, PostgreSQL, Oracle

NoSQL (Non-Relational)

- Unstructured / semi-structured data
- Flexible schema
- BASE (Basically Available, Soft state, Eventually consistent)
- Uses different query APIs
- Examples: MongoDB, Cassandra, Redis

Use Cases

Database Type	Use Cases
SQL	Banking, Finance, E-commerce (Transactions, Orders)
NoSQL	Real-time analytics, Logging, IoT, Content Management

Example

SQL Table (Users)

id	name	email	age
1	Aman	aman@email.com	28
2	Neha	neha@email.com	25

NoSQL Document (Users)

```
{
  "id": 1,
  "name": "Aman",
  "email": "aman@email.com",
  "age": 28,
  "skills": ["java", "sql"]
}
```

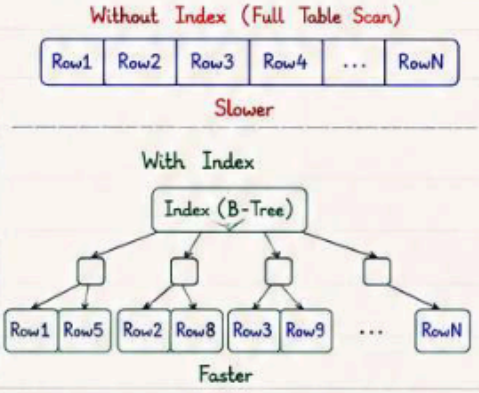
☆ Pro Tip : Choose the right database based on data structure, scalability needs and consistency requirements.

15. Database Indexing

Indexes improve the speed of data retrieval operations on a database table.

How it works :

1. DB creates an index on one or more columns
2. Index acts like a fast lookup table
3. DB uses index to find data quickly
4. Reduces full table scan



Example

```
CREATE INDEX idx_users_email
ON users(email);
```

Now, searching by email will be much faster.

Best Practices

- ✓ Create indexes on frequently searched columns
- ✓ Don't over-index (affects write performance)
- ✓ Use composite index if needed

☆ Pro Tip : Indexes speed up reads but slow down writes. Use only where it adds value.

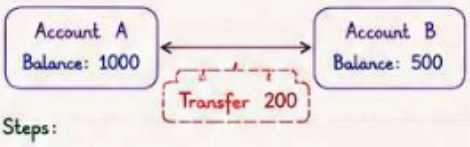
16. Transactions & ACID

Transactions ensure that a series of database operations are completed successfully.

ACID Properties

- ✓ Atomicity : All or nothing
- ✓ Consistency : Data stays valid
- ✓ Isolation : Transactions don't interfere
- ✓ Durability : Data is permanent

Example (Fund Transfer)



Steps:

1. Deduct 200 from Account A
2. Add 200 to Account B
3. If any step fails → rollback (No changes)
   If all steps success → commit

SQL Example

```
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 200
WHERE id = 1;
UPDATE accounts SET balance = balance + 200
WHERE id = 2;
COMMIT;  -- or ROLLBACK;
```

Remember

Without transactions, system may end up in inconsistent state during failure.

☆ Pro Tip : Always use transactions for critical operations like payments, transfers, order processing to avoid partial updates.

TOP 30 BACKEND CONCEPTS

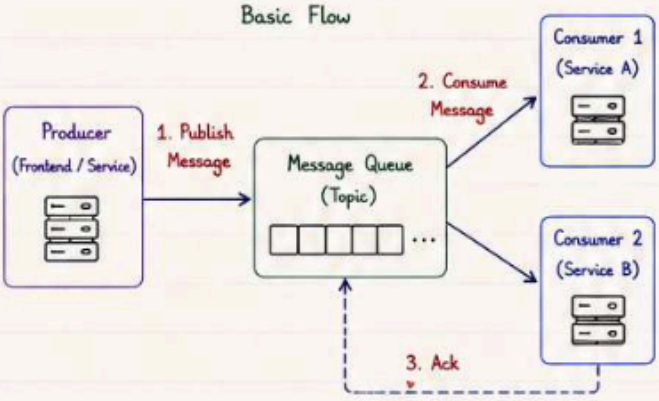
Every Developer Should Know These Concepts

17. Producer & Consumer

Producers send messages to a system (usually a message queue or topic) and consumers read and process those messages.

How it works :

1. Producer creates a message
2. Message is sent to a topic / queue
3. Consumer reads the message
4. Processes and acknowledges it



Example (Kafka)

Topic: orders

Producer → publishes order event

Consumer → processes order, updates DB, sends email

Benefits

- ✓ Decouples services
- ✓ Asynchronous processing
- ✓ Improves scalability
- ✓ Handles traffic spikes

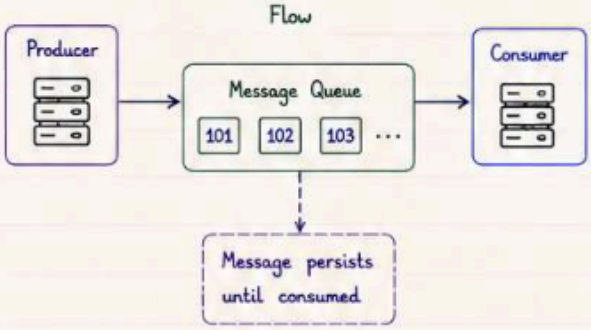
☆ Pro Tip : Keep messages small and idempotent. Acknowledge only after successful processing to avoid message loss.

18. Message Queue

A message queue stores messages until they are processed by consumers.

How it works :

1. Producer sends message to queue
2. Queue stores message safely
3. Consumer pulls message from queue
4. Message is removed / acknowledged after processing



Example (RabbitMQ / SQS)

- Order processing
- Email / Notification
- Report generation
- Background jobs

Queue Types

- FIFO Queue
- Priority Queue
- Delayed Queue
- Dead Letter Queue (DLQ)

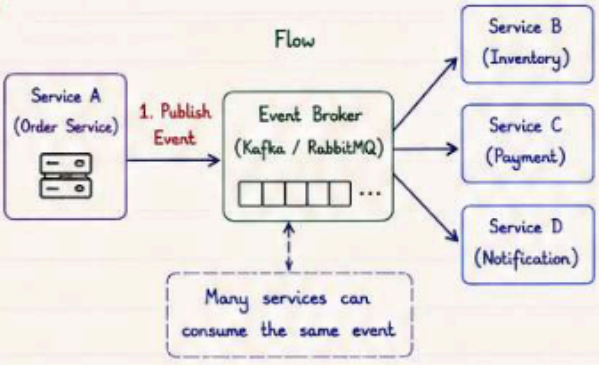
☆ Pro Tip : Use DLQ to handle failed messages and prevent reprocessing loops.

19. Event-Driven Architecture

In EDA, services communicate by producing and consuming events instead of direct requests.

How it works :

1. Service A produces an event
2. Event is published to the broker
3. Multiple services consume the event
4. Each service processes as needed



Example

When an order is placed → OrderPlaced event is published. Inventory, Payment, and Notification services react to it independently.

Benefits

- ✓ Loose coupling
- ✓ Scalability
- ✓ Real-time processing
- ✓ Easy to extend

☆ Pro Tip : Design events carefully with versioning. Events are contracts between services.

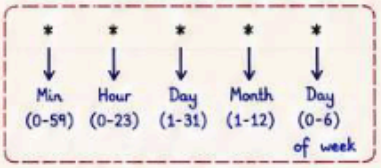
20. Scheduler / Cron Jobs

Schedulers run tasks automatically at a specific time or at regular intervals.

How it works :

1. Define a task
2. Set schedule (cron expression)
3. Scheduler triggers the task
4. Task runs in background

Cron Expression Format



Common Examples

- 0 0 * * * → Every day at 12:00 AM
- 0 */2 * * * → Every 2 hours
- 30 9 * * 1-5 → 9:30 AM on weekdays
- 0 0 1 * * → 1st day of every month

Example Use Cases

- Send daily summary emails
- Generate reports every night
- Clean up old logs
- Sync data every hour

Popular Tools

- Cron (Linux)
- Quartz Scheduler (Java)
- Spring @Scheduled
- Kubernetes CronJob

☆ Pro Tip : Always make scheduled jobs idempotent and handle failures with retries and alerts.

TOP 30 BACKEND CONCEPTS

Every Developer Should Know These Concepts

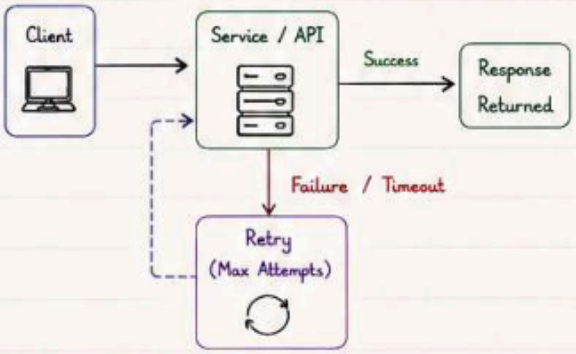
21. Retry Mechanism

Retry helps in handling temporary failures by retrying the request a few times before giving up.

How it works :

1. Request is sent to the service
2. If it fails (timeout/error) → retry
3. Retry with backoff (wait time)
4. If max retries reached → fail

Retry Flow



Example (Payment API)

- Payment request fails due to network issue
- Retry 3 times with exponential backoff
- If still fails → show error

Benefits

- ✓ Improves reliability
- ✓ Handles temporary failures
- ✓ Better user experience

★ Pro Tip : Use exponential backoff (e.g., 1s, 2s 4s, 8s) between retries to reduce load on the system.

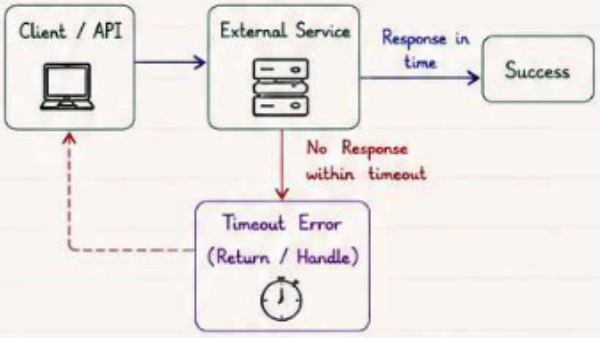
22. Timeout

Timeout ensures the system does not wait forever for a response from an external service.

How it works :

1. Send request to external service
2. Start timer
3. If response before timeout → success
4. If not → timeout error & handle

Timeout Flow



Example

- External service takes > 5s to respond
- Timeout is set to 5s
- System returns error and tries fallback / retry

Benefits

- ✓ Prevents hanging requests
- ✓ Improves system stability
- ✓ Allows fallback mechanism

★ Pro Tip : Set timeout based on service SLA. Too high → slow system, too low → unnecessary failures.

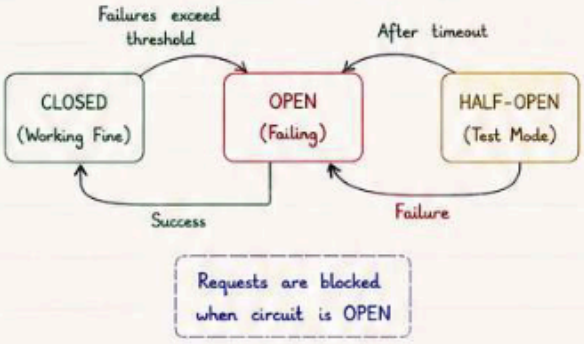
23. Circuit Breaker

Circuit Breaker prevents repeated failures by stopping requests to a failing service for some time.

How it works :

1. Requests are sent to service
2. If failures exceed threshold → open
3. Circuit opens → requests fail fast
4. After timeout → half-open → test
5. If success → close, else open again

Circuit Breaker States



Example

- Payment service is down
- After 5 failures → circuit opens
- Next requests fail fast
- After 30s → half-open → test
- If success → close the circuit

Benefits

- ✓ Prevents cascading failures
- ✓ Protects backend services
- ✓ Improves system resilience

★ Pro Tip : Circuit Breaker is a key pattern in microservices for building fault-tolerant systems.

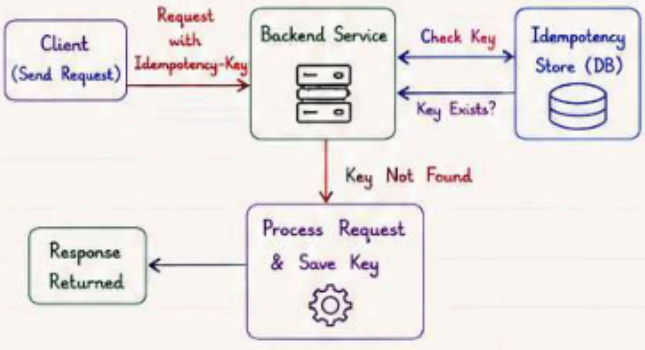
24. Idempotency

Idempotency ensures the same request can be sent multiple times without changing the result.

How it works :

1. Client sends request with Idempotency-Key
2. Server checks if key already exists
3. If yes → return previous result
4. If no → process and save key

Idempotency Flow



Example (Payment API)

- User clicks "Pay" multiple times
- Same Idempotency-Key sent
- Only one payment is created
- Subsequent requests return same response

Benefits

- ✓ Prevents duplicate actions
- ✓ Safe retries for APIs
- ✓ Essential for payments, orders

★ Pro Tip : Always use Idempotency-Key for operations that modify data (create charge, transfer, etc.).

TOP 30 BACKEND CONCEPTS

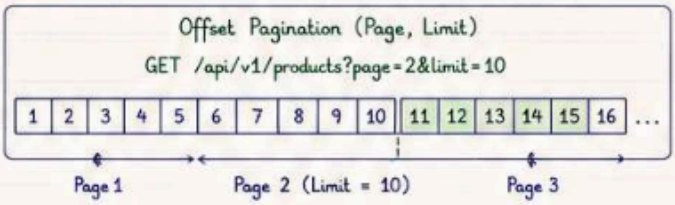
Every Developer Should Know These Concepts

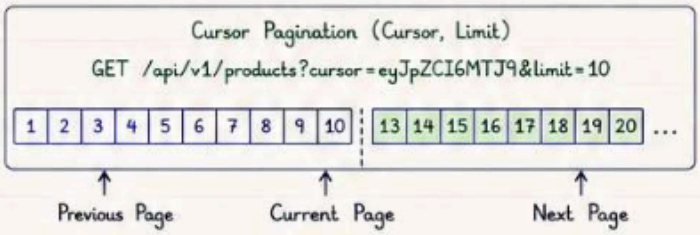
25. Pagination (Offset vs Cursor)

Pagination helps in splitting large datasets into smaller chunks for better performance and usability.

How it works :

1. Client requests a page
2. Server returns limited data
3. Client requests next page
4. Repeat until no data





Example Response

```
{
  "data": [ ... 10 items ... ],
  "page": 2,
  "limit": 10,
  "total": 120,
  "totalPages": 12,
  "hasNext": true,
  "hasPrev": true
}
```

Best Practice

- Use cursor pagination for real-time / large datasets
- Avoid high page numbers in offset pagination
- Always set a max limit
- to prevent abuse

Pro Tip : Cursor pagination is more efficient for large datasets and real-time feeds (e.g., chat, activity feed).

26. API Versioning

Versioning allows spdates to APIs without breaking existing clients.

Common Approaches :

- URI Versioning : /api/v1/users
- Query Parameter : /api/users?version=1
- Header Versioning : Accept: application/vnd.app.v1+json

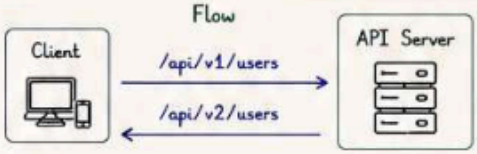
Why Versioning ?

- ✓ Maintain backward compatibility
- ✓ Safe API updates
- ✓ Gradual rollout of new features
- ✓ Better maintainability

Example

v1 → GET /api/v1/users
v2 → GET /api/v2/users
v3 → GET /api/v3/users

Flow



Pro Tip : Prefer URI versioning. It's easy to understand, cache-friendly and widely used.

27. Logging

Logging captures events, errors and debug information for monitoring and troubleshooting.

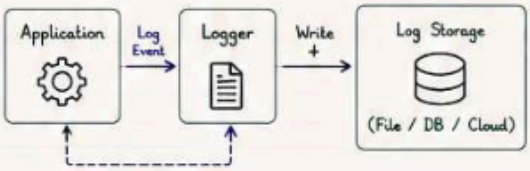
Best Practices

- Log meaningful messages
- Never log sensitive data
- Use appropriate log level
- Centralize logs (ELK, Loki, CloudWatch)

Log Levels

DEBUG – Detailed information for debugging
INFO – General operational information
WARN – Something unexpected but not harmful
ERROR – Something failed
FATAL – Critical failure, app may crash

Logging Flow



Example Log

```
2025-05-16 10:15:23 INFO  UserService - User created successfully. userId=123
2025-05-16 10:15:24 ERROR PaymentService - Payment failed. orderId=456, reason=Insufficient Funds
```

Pro Tip : Good logs reduce debugging time from hours to minutes. Log smart, not everything.

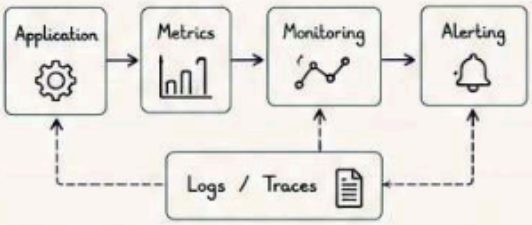
28. Monitoring & Observability

Monitoring tracks system health. Observability helps understand the WHY behind issues.

Key Components

- Metrics – Numeric data over time
- Logs – Detailed event data
- Traces – Request flow across services
- Alerts – Notify when something breaks

Observability Stack



Popular Tools

- Prometheus + Grafana (Metrics)
- ELK Stack / Loki (Logs)
- Jaeger / Zipkin (Tracing)
- PagerDuty / Opsgenie (Alerts)

Pro Tip : Set meaningful alerts. Too many alerts cause noise; right alerts save your system (and night!).

29. Health Checks

Health checks ensure your service is running and ready to handle traffic.

Health Check Types

Liveness Probe	Readiness Probe	Startup Probe
Is the service alive?	Is the service ready to serve?	Is the service fully started?

Use Cases

- Kubernetes probes
- Load balancer health check
- Auto restart unhealthy services

Pro Tip : Always implement health endpoints (e.g., /health/live, /health/ready) in every service.

TOP 30 BACKEND CONCEPTS

Every Developer Should Know These Concepts

30. Graceful Shutdown

Graceful shutdown allows a service to finish ongoing requests before shutting down. This prevents data loss and errors.

How it works :

① Shutdown signal received (SIGTERM)

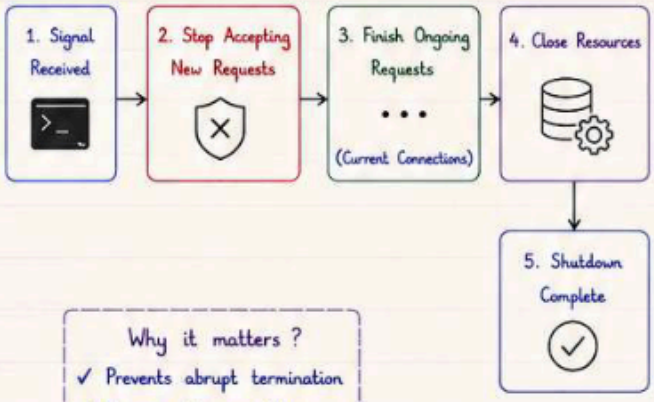
② Stop accepting new requests

③ Finish processing existing requests

④ Close connections & release resources

⑤ Shutdown complete

Graceful Shutdown Flow



Example (Spring Boot)

Add in application.properties :

server.shutdown=graceful

On SIGTERM :

- Finish active requests
- Close connections
- Release resources

Why it matters ?

- ✓ Prevents abrupt termination
- ✓ Ensures data consistency
- ✓ Better user experience

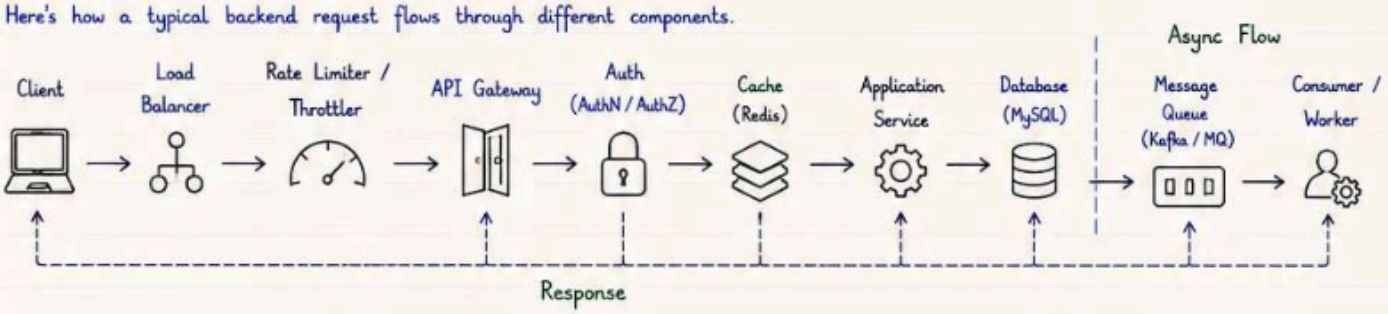
Best Practices

- Set a shutdown timeout
- Close DB connections properly
- Drain connections in Load Balancer
- Log shutdown events

Pro Tip : Always handle graceful shutdown in production systems to avoid data corruption and bad user experience.

Backend Request Journey (Putting it All Together)

Here's how a typical backend request flows through different components.



Key Takeaways

- ✓ Design APIs that are secure, scalable and reliable.
- ✓ Handle failures gracefully.
- ✓ Use caching, queues and async processing for performance.
- ✓ Monitor, log and improve continuously.
- ✓ Always think about user impact and system resilience.

Backend Interview Quick Cheat Sheet

- REST, HTTP, Status Codes
- Auth, AuthZ, Tokens
- Pagination, Filtering, Sorting
- Caching (Redis)
- Rate Limiting & Throttling
- Load Balancer
- DB Connection Pooling
- SQL vs NoSQL
- Indexing
- Transactions (ACID)
- Producer / Consumer
- Message Queue
- Event Driven Architecture
- Scheduler / Cron Jobs
- Retry Mechanism
- Timeout
- Circuit Breaker
- Idempotency
- Horizontal vs Vertical Scaling
- Stateless vs Stateful
- Distributed Systems
- Service Discovery
- Logging
- Monitoring & Observability
- Health Checks
- Graceful Shutdown

Remember

Backend is not just about writing APIs.

It's about building systems that are fast, reliable, scalable and user-friendly.

Final Words

Master these 30 backend concepts and you will be able to design, build and maintain robust backend systems like a pro! Keep learning. Keep building. Keep improving!